

ASSIGNMENT - 1

LINUX COMMANDS - SYNTAX, PURPOSE, EXAMPLES

1. pwd: print working directory

purpose: shows the current directory you are in.

syntax: `$pwd /home/user`

2. Mkdir: Make directory

purpose: Creates folders

`mkdir my-folder`

3. cd: change Directory.

purpose: Moves between folders

`cd home`

4. ls: List

purpose: List files and folders in the current directory.

`ls [options]`

5. cp-copy.

purpose: copies files or folders

`cp source destination`

6. mv - Move or Rename

purpose: Moves or renames

`mv source destination`

7. touch - Create Empty File

purpose - creates a new empty path

`touch filename.`

8. echo - Display Text

Purpose - prints text to the screen or into a file.

echo "message"

9. cat - Concatenate and Display file

Purpose - shows file contents

cat filename

10. df - Disk Free

Purpose - shows disk space usage

df [options].

11. du - Disk usage

Purpose - shows how much space a file or folder takes.

du [options] path.

12. head - First lines of a file

Purpose - shows the first few lines of a file.

head filename.

13. tail - Last lines of a file

Purpose - shows the last few lines

tail filename.

14. clear - Clear screen

Purpose - clears the terminal display.

clear.

15. tar - Archive files

Purpose: creates or extracts compressed archive files.

-br options archive_files

16. cal - calendar

Purpose: displays a calendar.

cal [month] [year]

17. date - show date & time

Purpose: displays the systems date and time.

date

18. chmod - change file permissions

Purpose: changes who can read/write.

chmod permissions filename.

19. wc - word count

Purpose: counts lines

wc filename

20. Banner

Purpose: shows the enlarged text on the screen

SYNTAX: `banner <text>`

ASSIGNMENT-2

1. BIGGEST OF 3 NUMBERS:

```
read -p 'Enter three Numbers' a b c
```

```
if [ $a -ge $b ] && [ $a -ge $c ]; then
```

```
    echo '$a is the biggest'
```

```
elif [ $b -ge $a ] && [ $b -ge $c ]; then
```

```
    echo '$b is biggest'
```

```
else  
    echo '$c is biggest'
```

```
fi
```

OUTPUT:

1 2 3

3

2. Factorial of Number

```
read -p 'Enter a number:' n
```

```
f = 1
```

```
for ((i=1; i<=n; i++)); do
```

```
    f = $((f*i))
```

```
done
```

```
echo 'Factorial = $f'
```

OUTPUT:

5

120

3. Sum of First n Natural Numbers (n=12)

```
n=12
```

```
sum=0
```

```
for ((i=1; i<=n; i++)); do
```

```
    sum = $((sum+i))
```

```
done
```

```
echo 'sum = $sum'
```

OUTPUT:

12

78

4. Pattern Printing:

```
for((i=5; i>=1; i--)); do
  for ((j=1; j>=1; j--)) : do echo -n '$j' ; done
  for ((k=5; k>=1; k--)) ; do echo -n 'x' ; done
  echo
done
```

OUTPUT:

```

*
* *
* * *
* * * *
* * * * *
```

5. Swap Two Numbers using Function

```
swap() {
  temp=$1 ; a=$2 ; b=$temp
  echo "After swap: a=$a b=$b"
}
read -p "Enter a and b: " a b
Swap $a $b.
```

OUTPUT

a=2, b=3

a=3, b=2

ASSIGNMENT - 3 :

1. FORK() system call

```
#include <unistd.h>
```

```
#include <stdio.h>
```

```
#include <stdio.h>
```

```
void main()
```

```
{  
    int pid;
```

```
    pid = fork();
```

```
    if (pid != 0)
```

```
    {  
        printf("In child process id %d", getpid());
```

```
        printf("In I am a child, my parent process is %d",  
                getpid());
```

```
    }
```

```
}
```

Output :

child process id is 3456

I am a child, my parent process is 3455



2. STAT C) system call

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```
int main() {
```

```
    int status;
```

```
    status = system('stat prog1.c');
```

```
    if (status == -1) {
```

```
        perror("system call failed");
```

```
        return 1;
```

```
    }
```

```
    return 0;
```

```
}
```

output: 1



3. WAIT(), EXIT(), EXEC() system call:

```
#include <sys/wait.h>
```

```
#include <unistd.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    pid_t pid = fork();
```

```
    if (pid == 0) {
```

```
        exec ("/bin/ls", "ls", "-l", NULL);
```

```
        perror("exec failed");
```

```
        exit(1);
```

```
    }
```

```
    else if (pid > 0) {
```

```
        wait(NULL);
```

```
        printf("child process finished\n");
```

```
    } else {
```

```
        perror("(n fork failed)");
```

```
        return 1;
```

```
    }
```

```
    return 0;
```

```
}
```

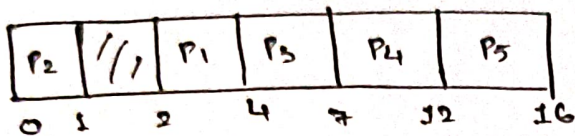
output: child process finished.

Experiment - 2:

1. FCFS (First come First Served)

Process-ID	Arrival time	CPU (Burst time)
P ₁	2	2
P ₂	0	1
P ₃	2	3
P ₄	3	5
P ₅	4	4

A. Gantt chart



Ready Queue:



Observations:

Process-ID	AT	BT	ST	CT	TAT	WT	RT
P ₁	2	2	2	4	2	0	0
P ₂	0	1	0	1	1	0	0
P ₃	2	3	4	7	5	2	2
P ₄	3	5	7	12	9	4	4
P ₅	4	4	12	16	12	8	8

$$TAT = CT - AT$$

$$WT = TAT - BT$$

$$RT = ST - AT$$

$$CT = ST + BT$$

CONCLUSION

a) Average TAT = $29/5 = 5.8$

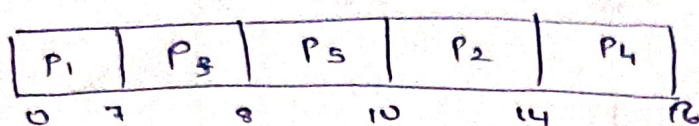
b) Average WAT = $14/5 = 2.8$

c) Average RT = $14/5 = 2.8$

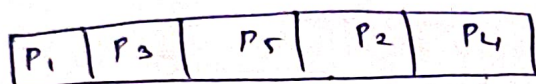
2. SJF (Shortest Job First)

Process ID	Arrival Time	CPU (B.T)	ST	CT	TAT	WT
P ₁	0	7	0	7	7	0
P ₂	2	4	7	11	9	5
P ₃	4	1	11	12	8	7
P ₄	5	4	12	16	11	7
P ₅	6	2	16	18	12	10

Gantt chart



Ready Queue



OBSERVATIONS

→ Refer above table

→ $TAT = CT - AT$

$WT = TAT - BT$

$RT = ST - AT$

$CT = ST + BT$

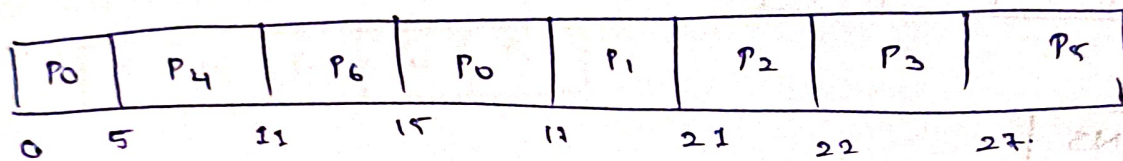
CONCLUSION

- Average Turnaround Time = 8.00
- Average Waiting Time = 4.40
- Average Response Time = 4.40

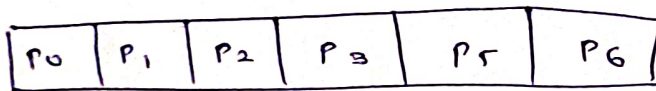
3. preemptive priority Scheduling

process ID	AT	BT	priority	CT	TAT	WT
P ₀	0	8	3	15	15	7
P ₁	1	2	4	17	16	14
P ₂	3	4	4	21	18	14
P ₃	4	1	5	22	18	17
P ₄	5	6	2	12	7	1
P ₅	6	5	6	22	21	16
P ₆	10	1	1	11	7	0

Gantt chart



Ready Queue



OBSERVATIONS

→ Refer above table

$$TAT = CT - AT$$

$$WT = TAT - BT$$

$$RT = ST - AT$$

$$CT = ST + BT$$

CONCLUSION

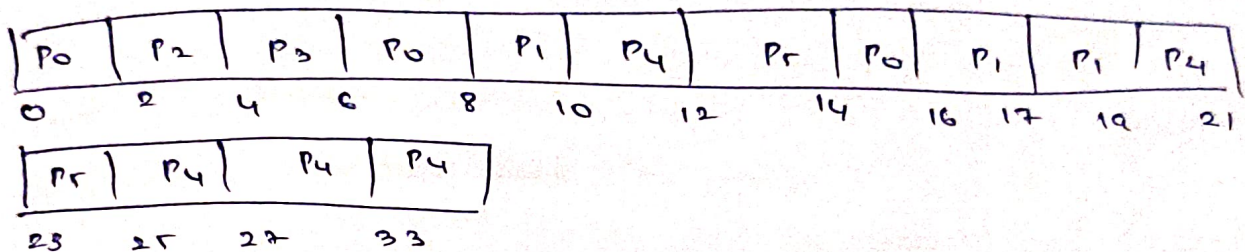
→ Average Turnaround Time = 18.21

→ Average Waiting Time = 9.86

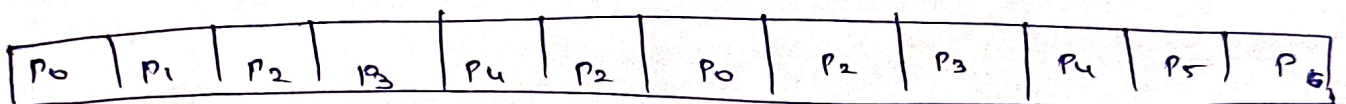
4. Round Robin (Time Quantum: 2)

process - ID	AT	BT	CT	TAT	WT
P ₀	0	6	17	17	11
P ₁	1	8	22	26	19
P ₂	2	2	6	4	2
P ₃	3	1	9	6	5
P ₄	4	12	33	29	17
P ₅	6	4	23	17	13

Ganttchart



Ready Queue



OBSERVATION

→ Refer above table

$$TAT = CT - AT$$

$$WT = TAT - BT$$

$$RT = ST - AT$$

$$CT = ST + BT$$

CONCLUSION

→ Average Turnaround Time = 16.50

→ Average Waiting Time = 11.00

[Signature]
24/9/22

Experiment NO: 3

Title : Implementation of Multithreading in Java and PThreads

Aim : To study thread concepts, thread functions, and implement multithreading using Java and PThreads (POSIX threads in C)

Objectives :

1. To understand the concept of threads.
2. To study types and uses of threads.
3. To implement thread function in java.
4. To implement thread creation and synchronization in PThreads (C).

Theory : 1. Theory Notes:

1. Definition of Thread:

- A thread is the smallest unit of a process that can be scheduled and executed independently.
- Multiple threads in a process share the same memory space but execute different parts of the program concurrently.

2. Types of Threads:

1. User-level threads - Managed by user libraries, not visible to the OS kernel.
2. Kernel-level threads - Managed directly by the operating system kernel.
3. Java Threads - Implemented using Thread class or Runnable interface.
4. POSIX Threads (PT Threads) - Portable standard threads in C/C++ for UNIX/Linux systems.

3. Uses of Threads:

- parallel execution of tasks.
- Efficient CPU utilization.
- Faster program execution.
- useful in operating systems, GUI applications, servers

4. Thread Functions:

Java Thread Function

-) Start() → starts a thread
-) run() → Defines the thread's task
-) sleep(ms) → pauses thread temporarily.
-) yield() → pauses to let other threads execute
-) Join() → waits for a thread to finish
-) wait() → Makes thread wait until notified.
-) notify() / notifyAll() → wakes up waiting threads.

PThread Functions(c)

-) pthread_create() → creates a new thread
-) pthread_exit() → Terminates calling thread.
-) pthread_join() → Waits for a thread to finish
-) pthread_mutex_lock() → Locks a critical section.
-) pthread_cond_wait() → Waits for a condition variable
-) pthread_cond_signal() → Signals a waiting thread.

PROGRAMS → Attached

Result:

-) studied concepts of threads, types, uses and thread functions.
-) Implemented thread functions in java (sleep, yield, join, wait, notify)
-) Implemented PThreads in c (pthread_create, pthread_join, etc)
-) Successful executed programs and observed output.


24/9/21

EXPERIMENT-4 IMPLEMENTATION OF IPC

Aim

: To study and implement different ipc mechanisms using semaphores and pipes: producer-consumer, Reader-Writers, dining philosophers and pipes.

Objectives

- 1. To understand Synchronization in multiprocess system
- 2. To avoid race conditions, deadlocks and starvation.
- 3. To implement ipc using semaphores and pipes

Theory :-

- 1. IPC enables processes to exchange data and continue actions.
- 2. producer-consumer problem :- A shared buffer is accessed by producer and consumer process.
- 3. Readers-Writers problem: Multiple readers can read simultaneously but writers need exclusive access.
- 4. dining philosopher: philosophers share chopsticks to eat. Semaphores prevent deadlock.
- 5. Pipes provide a unidirectional communication channel between processes. Data written by one process can be read by another.

Result

:- This experiment demonstrated safe and efficient communication among concurrent process.

Aim : To study and Implement Banker's Algo for deadlock avoidance and memory management techniques using paging and Segmentation

Objectives :

- To ensure safe resource allocation using Banker Algo.
- To implement paging for efficient memory
- To implement Segmentation based on user program structure

Theory : a) Banker Algorithm;

used for deadlock avoidance. It checks if resource allocation to a process leads to a safe state. processes request resources and the system grants only if it does not cause deadlock.

b) paging;

paging is a memory management scheme that eliminates external fragmentation.

Logical memory is divided into fixed blocks called frames.

c) Segmentation.

Segmentation divides memory into variable-sized segments as per program structure. Each supports logical division of programs but may cause external frag.

Calculations: A) Banker Algo:

Need				Allocation				Max				Available			
R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄	R ₁	R ₂	R ₃	R ₄
0	0	4	0	P ₀	2	0	1	2	2	0	5	2	4	5	2 (3)
0	2	5	0	P ₁	1	0	0	0	1	7	5	0	8 (1)	5 (5)	3 4 (1)
1	0	0	2	P ₂	1	3	5	4	2	3	5	6	7 (5)	5 (4)	3 4
0	0	2	0	P ₃	0	6	3	2	0	6	5	2	7 (2)	8 (6)	8 8

P₁ → P₃ → P₀ → P₂.

$$\text{Need} = \text{Allocation} - \text{Max}$$

B) Memory Management using paging.

- * Logical Memory Size = 64 KB
- * physical Memory size = 128 KB
- * Frame size = 8 KB
- * page (Number) = 2
- * Offset = 200 bytes
- * Frame size = 8192 bytes

$$\begin{aligned} \text{PA} &= (\text{Frame Number} \times \text{Frame Size}) + \text{offset} \\ &= (7 \times 8192) + 300 = 57644 \text{ bytes} \end{aligned}$$

c) Implementation of Memory Management. Using Segments

Segment Num	Base Address	Limit
0	1000	5000
1	20000	100000
2	30000	30000

Ex: Segment Num = 1 offset = 100 ⇒ 20100

EXPERIMENT-6

Aim : To implement and study different page replacement algorithm in java and calculate total page faults, page hits, hit ratio, and fault ratio.

Objective : To understand how operating systems manage limited physical memory using page replacement algorithms and how these methods help minimize page faults for efficient memory.

Theory : When a process tries to access a page not present in main memory, a page fault occurs. The operating system must then bring the required page from secondary storage. If no free frame exists, a page replacement algo selects a page to remove from memory.

Types of Page Replacement Algorithms.

1) FIFO (First-In-First-out)

The oldest loaded page is replaced first, simple but may lead to inefficiency

2) LRU (Least recently used)

The page that hasn't been used for a longest time is replaced, works well in practice.

3) optimal :

Replaces the page that will not be used for the longest time in the future.

Stepwise Mathematical procedure + (FIFO)

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Frames: 3

Step	Page	Frames	Fault
1	1	1	Fault
2	2	1 2	Fault
3	3	1 2 3	Fault
4	4	2 3 4	Fault
5	1	3 4 1	Fault
6	2	4 1 2	Fault
7	5	1 2 5	Fault
8	1	1 2 5	Hit
9	2	1 2 5	Hit
10	3	2 5 3	Fault
11	4	5 3 4	Fault
12	5	3 4 5	Fault

Total page Fault: 10

Total Hits: 2

Hit Ratio: 0.166

Fault Ratio: 0.833

Stepwise Mathematical procedure (sample: optimal)

Reference string : 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 2, 2, 3

Frames : 4

Step	page	Frames	Decision
1	7	7	Fault
2	0	7 0	Fault
3	1	7 0 1	Fault
4	2	7 0 1 2	Fault
5	0	7 0 1 2	Hit
6	3	3 0 1 2	Replace 7
7	0	3 0 1 2	Hit
8	4	3 0 1 2	Replace 1
9	2	3 0 4 2	Hit
10	3	3 0 4 2	Hit
11	0	3 0 4 2	Hit
12	3	3 0 4 2	Hit
13	2	3 0 4 2	Hit
14	2	3 0 4 2	Hit

Total page Faults : 6

Total Hits : 8

Hit ratio : 0.571

Fault ratio : 0.429

Conclusion :

→ The page replacement algorithms help efficiently manage memory.

→ LRU provides a balance between simplicity and effectiveness